



网络安全创新创业实践课实验报告

Bitcoin 密码算法的安全修复与性能优化

第 41 组

2026 年 7 月 27 日

小组成员及分工

本实验由小组成员共同完成。姜良萍、李雨晴主要负责常数时间安全修复部分，王奕文、曾静雯主要负责固定基点标量乘法性能优化部分。

表 1: 小组成员及分工

姓名	学号	班级	负责模块	对应章节
姜良萍（组长）	202300460098	网安 1 班	负责实验总体方案与安全修复实验的工程框架；管理仓库版本、Git worktree、构建目录及运行脚本；整理实验命令；负责全文整合、格式统一、总结撰写与最终复核。	第 1 章；第 2.1、2.2 节；第 3.1 节；第 4.3 节；第 6 章；附录 A.1–A.6
李雨晴	202300460025	网安 2 班	负责常数时间漏洞复现与修复验证；搭建 Clang 15、Valgrind 和 ctime_tests 实验环境；执行修复前后常数时间测试和功能测试；分析错误日志、调用栈、补丁差异及安全影响。	第 4.1、4.2、4.4 节；协助核对第 6 章安全修复结论
王奕文	202300460056	密码 1 班	负责固定基点标量乘法性能优化的源码与数学原理分析；研究旧固定宽度预计算算法和 signed-digit multi-comb 算法的标量分解、查表结构、点加与倍点运算量及安全-性能权衡。	第 5.1、5.2、5.3 节；第 5.5 节中的算法原理与安全-性能分析；协助核对第 6 章性能结论
曾静雯	202300460045	网安 1 班	负责固定基点标量乘法性能实验；构建不同预计算表及跨版本配置，设计并执行交错重复 benchmark；统计时延、置信区间、性能提升和文件大小，绘制图表并完成实验结果分析。	第 2.3、2.4 节；第 3.2 节；第 5.4 节；第 5.5 节中的结果分析与实验局限性；附录 A.7–A.9

目录

1 实验目的	1
2 实验环境	1
2.1 安全修复实验环境	1
2.2 安全修复构建选项	2
2.3 性能优化实验环境	2
2.4 性能优化实验构建选项	3
3 实验总体设计	3
3.1 安全修复实验分组	3
3.2 性能优化实验分组	3
4 常数时间安全问题复现	4
4.1 secp256k1 中的秘密标量运算与常数时间防护	4
4.2 条件移动缺陷的根因与数学原理	6
4.3 实验流程	7
4.4 结果讨论与安全影响	17
5 固定基点标量乘法性能优化	18
5.1 优化对象与版本变化	18
5.2 新旧固定基点标量乘法算法	19
5.3 性能实验设计	22
5.4 实验结果	25
5.5 结果分析与讨论	29
6 总结	32
A 主要实验命令	32
A.1 建立工作区	32
A.2 构建修复前版本	32
A.3 构建修复后版本	33
A.4 执行常数时间测试	33
A.5 执行普通功能测试	33
A.6 查看补丁	34
A.7 建立性能实验工作区	34
A.8 构建六种性能配置	35
A.9 执行性能基准测试	37

1 实验目的

本实验的总体目标是阅读 libsecp256k1 仓库中的真实代码修改，理解 Bitcoin 密码算法在工程实现中面临的安全性及性能问题，并通过编译、功能测试、常数时间检测和性能基准测试，验证相关修改的必要性与有效性。具体目标如下：

1. 了解 libsecp256k1 在 Bitcoin 密钥生成、ECDSA 签名、Schnorr 签名和 ECDH 等密码操作中的作用；
2. 复现 Clang 15 优化导致条件移动失去常数时间性质的安全问题；
3. 分析修复前后 cmov 实现的代码差异、布尔代数原理及其安全影响；
4. 使用 Valgrind 跟踪秘密数据，验证秘密是否影响程序控制流，并区分“功能正确”与“实现安全”；
5. 分析固定基点标量乘法函数 ecmult_gen 的算法更新及其数学原理；
6. 比较不同预计算表配置和新旧算法的运行性能与存储开销，评价其时间-空间权衡。

2 实验环境

2.1 安全修复实验环境

实验环境如表 2 所示。

表 2: 实验环境

项目	配置
操作系统	Ubuntu 24.04.4 LTS, 运行于 WSL2
Linux 内核	6.6.87.2-microsoft-standard-WSL2
CPU	13th Gen Intel Core i7-13700H
分配逻辑 CPU	6
指令集架构	x86_64
编译器	Clang 15.0.7
Valgrind	3.22.0
CMake	3.28.3
Git	2.43.0
修复前提交	464a9115b4eda46b464d22829ece4f51985944bf
修复后提交	4a496a36fb07d6cc8c99e591994f4ce0c3b1174c

2.2 安全修复构建选项

修复前后版本均使用以下关键选项：

```
cmake \  
-S <source-dir> \  
-B <build-dir> \  
-DCMAKE_C_COMPILER=clang-15 \  
-DSECP256K1_BUILD_CTIME_TESTS=ON \  
-DSECP256K1_VALGRIND=ON \  
-DCMAKE_BUILD_TYPE=None \  
-DCMAKE_C_FLAGS="-O1"
```

其中：

- SECP256K1_BUILD_CTIME_TESTS=ON 用于构建常数时间测试程序；
- SECP256K1_VALGRIND=ON 启用 Valgrind 客户端请求，使测试程序可以主动标记秘密数据；
- -O1 触发本次研究的 Clang 优化行为；
- 两个版本保持完全相同的编译器和优化选项。

2.3 性能优化实验环境

固定基点标量乘法性能实验使用的环境如表 3所示。

表 3: 性能优化实验环境

项目	配置
处理器	Intel Core i5-13500H
操作系统	WSL2 Linux, x86_64
编译器	GCC 13.3.0
CMake	3.28.3
Ninja	1.11.1
构建模式	Release
库类型	静态库
CPU 绑定	逻辑 CPU 0
benchmark 迭代数	50000
外部重复次数	12

2.4 性能优化实验构建选项

性能实验统一采用 Release 模式、静态库和关闭汇编优化的配置，以尽量隔离固定基点标量乘法算法本身带来的性能变化。

3 实验总体设计

本实验报告包括两个相互关联的案例：

1. **安全修复案例：**复现并分析提交 4a496a3 对 Clang 15 常数时间问题的修复；
2. **性能优化案例：**分析 v0.5.0 引入的 signed-digit multi-comb 固定基点标量乘法算法，并通过不同表大小和跨版本基准测试验证其性能改进。

前者关注密码实现是否保持秘密无关的执行轨迹，后者关注在保持常数时间查表设计的前提下，如何改善签名和公钥生成的时间-空间权衡。

3.1 安全修复实验分组

为保证对比严谨，实验没有直接选择差异较大的两个发行版本，而是围绕修复提交建立两个工作区：

表 4: 安全修复实验分组

实验组	源代码版本	说明
修复前	4a496a3 [^] ，完整提交为 464a9115...	补丁应用前的父提交
修复后	4a496a36...	应用常数时间修复补丁

两组均使用 Clang 15.0.7、-O1、Valgrind 和相同 CMake 选项。这样可以尽量将结果差异归因于补丁本身。

3.2 性能优化实验分组

性能优化案例包含两组实验：

1. 固定 v0.5.1，比较 2 KiB、22 KiB 和 86 KiB 三种 `ecmult_gen` 预计算表；
2. 比较 v0.4.1 旧固定宽度算法的 32、64、512 KiB 配置，与 v0.5.0 新 multi-comb 算法的 2、22、86 KiB 配置。

4 常数时间安全问题复现

4.1 secp256k1 中的秘密标量运算与常数时间防护

4.1.1 libsecp256k1 与 Bitcoin

libsecp256k1 是面向 secp256k1 椭圆曲线的高性能 C 语言密码库。Bitcoin 使用 secp256k1 曲线完成密钥生成和数字签名等操作。设曲线基点为 G ，其阶为 n ，私钥为

$$d \in \mathbb{Z}_n^*,$$

则公钥为

$$P = dG.$$

在 ECDSA 签名过程中，签名者需要生成秘密随机数或确定性随机数 k ，并计算

$$R = kG = (x_R, y_R),$$

$$r = x_R \bmod n,$$

$$s = k^{-1}(e + rd) \bmod n,$$

其中 e 是消息摘要对应的整数。因此，私钥 d 和签名随机数 k 都属于必须严格保护的秘密数据。

4.1.2 椭圆曲线标量乘法中的秘密选择

标量乘法可以从二进制展开理解。设

$$k = \sum_{i=0}^{255} k_i 2^i, \quad k_i \in \{0, 1\},$$

则

$$kG = \sum_{i=0}^{255} k_i (2^i G).$$

在实际优化实现中，程序通常将标量划分为若干窗口，并根据每个秘密窗口值从预计算点表中选择相应点。为了防止秘密值通过内存访问地址或条件分支泄露，程序不能直接执行

```
1 if (secret_flag) {  
2     result = a;  
3 } else {  
4     result = r;  
5 }
```

而应采用不依赖秘密分支和秘密索引的条件移动。

4.1.3 常数时间实现的安全目标

密码实现中的“常数时间”并不简单等价于每次运行的物理时间绝对一致，而是要求攻击者能够观察到的关键执行特征不依赖秘密值。理想情况下，对任意公开输入 x 和任意两个秘密 s_1, s_2 ，应有

$$\text{Trace}(P, x, s_1) = \text{Trace}(P, x, s_2),$$

其中执行轨迹 Trace 至少包括：

- 条件控制流；
- 内存访问地址；
- 与秘密相关的查表位置；
- 可能被时间、缓存或分支预测观测到的执行差异。

若某个分支条件依赖私钥或 nonce，则攻击者可能通过重复测量建立统计模型。例如，可将执行时间简化表示为

$$T = T_0 + \Delta b + \varepsilon,$$

其中 $b \in \{0, 1\}$ 为秘密比特， Δ 为秘密相关分支造成的系统性时间差， ε 为系统噪声。当测量次数增加时，噪声可能部分抵消，而秘密相关差异可能被统计分析识别。

4.1.4 条件移动的布尔代数原理

设 $\text{flag} \in \{0, 1\}$ ，条件移动希望实现

$$r' = \begin{cases} r, & \text{flag} = 0, \\ a, & \text{flag} = 1. \end{cases}$$

对于 w 位无符号整数，可以构造掩码

$$m_0 = \text{flag} - 1 \pmod{2^w}, \quad m_1 = \neg m_0.$$

当 $\text{flag} = 0$ 时，

$$m_0 = 2^w - 1, \quad m_1 = 0;$$

当 $\text{flag} = 1$ 时，

$$m_0 = 0, \quad m_1 = 2^w - 1.$$

因此

$$r' = (r \wedge m_0) \vee (a \wedge m_1)$$

能够在 C 源代码中不使用显式 `if` 的情况下完成条件选择。

但是源代码中没有分支，并不等于最终机器代码中没有分支。编译器可能识别该表达式的条件选择语义，并将其重新优化成依赖 `flag` 的跳转或条件内存访问。

4.2 条件移动缺陷的根因与数学原理

4.2.1 修复前的条件移动代码

以 64 位有限域实现中的 secp256k1_fe_cmov 为例，修复前核心代码为：

```

1 static SECP256K1_INLINE void secp256k1_fe_cmov(
2     secp256k1_fe *r,
3     const secp256k1_fe *a,
4     int flag
5 ) {
6     uint64_t mask0, mask1;
7     SECP256K1_CHECKMEM_CHECK_VERIFY(r->n, sizeof(r->n));
8     mask0 = flag + ~((uint64_t)0);
9     mask1 = ~mask0;
10    r->n[0] = (r->n[0] & mask0) | (a->n[0] & mask1);
11    r->n[1] = (r->n[1] & mask0) | (a->n[1] & mask1);
12    /* 其余 limb 同理 */
13 }

```

在 64 位无符号整数模 2^{64} 的意义下，

$$\sim 0 = 2^{64} - 1 \equiv -1 \pmod{2^{64}},$$

因此

$$\text{mask0} = \text{flag} - 1 \pmod{2^{64}}.$$

当 $\text{flag} = 0$ 时，

$$\text{mask0} = 2^{64} - 1, \quad \text{mask1} = 0;$$

当 $\text{flag} = 1$ 时，

$$\text{mask0} = 0, \quad \text{mask1} = 2^{64} - 1.$$

于是对每个 64 位 limb 有

$$r'_i = (r_i \wedge \text{mask0}) \vee (a_i \wedge \text{mask1}),$$

从而实现

$$r' = \begin{cases} r, & \text{flag} = 0, \\ a, & \text{flag} = 1. \end{cases}$$

4.2.2 源码无分支不等于机器码无分支

上述 C 代码从语法上没有 if。然而，Clang 15 能够识别整个掩码表达式的高层语义，即“根据 flag 在 r 与 a 之间选择”。在优化阶段，编译器可能将其还原成条件跳转或秘密相关内存选择。

因此，漏洞根因不是数学计算结果错误，而是：

原本为避免秘密相关控制流而设计的位掩码条件移动，在特定编译器优化后重新产生了依赖秘密条件的机器级控制流。

普通功能测试只检查最终输出是否正确，而不会检查编译后的执行轨迹是否依赖秘密。Valgrind 的动态秘密跟踪恰好暴露了这一差异。

4.2.3 1024 次触发的算法解释

本次构建配置中，固定基点乘法精度参数为 4 位。一个 256 位标量可以划分为

$$\frac{256}{4} = 64$$

个窗口。若每个窗口通过逐项条件移动扫描

$$2^4 = 16$$

个候选表项，则一次固定基点乘法中的条件移动次数为

$$64 \times 16 = 1024.$$

该数量与公钥生成、ECDSA 签名、Schnorr 签名、密钥对创建和上下文随机化等路径中观察到的 1024 次动态触发相吻合。因此，错误计数能够从窗口化标量乘法及其常数时间查表结构得到一致解释。

4.3 实验流程

4.3.1 建立修复前后对照工作区

首先克隆 libsecp256k1 仓库，并以修复提交 4a496a3 的父提交和该提交本身建立两个相互独立的工作区：

```
git clone https://github.com/bitcoin-core/secp256k1.git
cd secp256k1

git worktree add ../secp-before '4a496a3~'
git worktree add ../secp-after 4a496a3
```

其中：

- secp-before 对应修复前提交 464a9115b4eda46b464d22829ece4f51985944bf；
- secp-after 对应修复提交 4a496a36fb07d6cc8c99e591994f4ce0c3b1174c。

通过这种方式，可以保证两个实验版本之间的主要差异就是本次常数时间修复补丁，减少其他代码修改对实验结果的干扰。

4.3.2 构建修复前版本

修复前版本使用 Clang 15.0.7 和 -O1 优化等级进行构建，同时开启 Valgrind 支持和常数时间测试程序：

```
cmake \  
  -S secp-before \  
  -B build-before \  
  -DCMAKE_C_COMPILER=clang-15 \  
  -DSECP256K1_BUILD_CTIME_TESTS=ON \  
  -DSECP256K1_VALGRIND=ON \  
  -DCMAKE_BUILD_TYPE=None \  
  -DCMAKE_C_FLAGS="-O1"  
  
cmake --build build-before -j"${nproc}"
```

其中：

- SECP256K1_BUILD_CTIME_TESTS=ON 用于构建 ctime_tests；
- SECP256K1_VALGRIND=ON 用于启用 Valgrind 客户端请求；
- -O1 用于触发本实验所分析的 Clang 优化行为；
- CMAKE_BUILD_TYPE=None 用于避免 CMake 额外添加其他优化参数。

构建完成后，生成了 ctime_tests、普通功能测试程序、共享库和静态库。

4.3.3 常数时间测试机制

ctime_tests 并不是将真正未初始化的变量传入密码算法，而是先生成合法测试输入，再通过 Valgrind 客户端请求，将私钥、签名 nonce 或盲化因子对应的内存主动标记为“未定义”。

这种处理相当于为秘密数据添加动态污点。若被标记的数据只参与普通算术运算，Valgrind 不一定报告错误；若秘密数据进一步影响条件跳转或可观察的内存访问，Valgrind 会报告：

```
Conditional jump or move depends on uninitialised value(s)
```

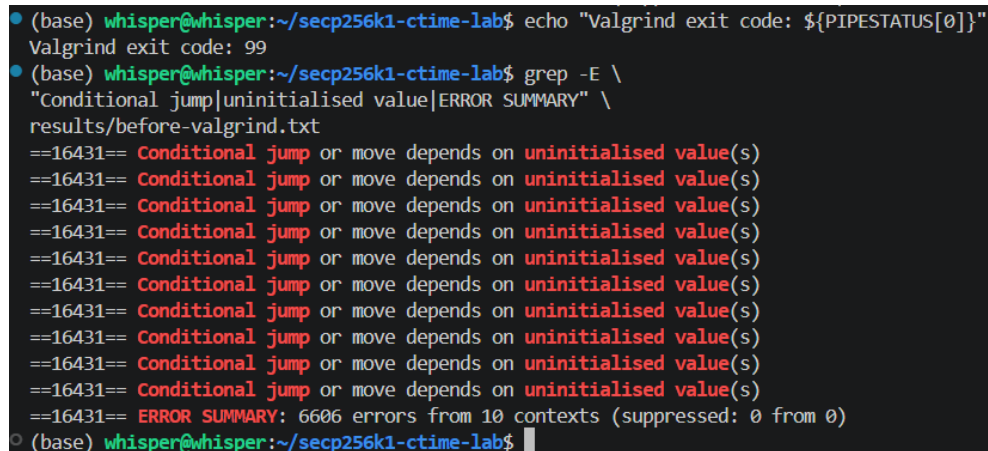
因此，本实验中的 uninitialised value 表示被测试程序标记为秘密的数据传播到了条件控制流。

4.3.4 执行修复前常数时间检测

修复前版本使用以下命令执行 Valgrind 检测：

```
valgrind \  
  --tool=memcheck \  
  --error-exitcode=99 \  
  --track-origins=yes \  
  --num-callers=20 \  
  ./build-before/src/ctime_tests \  
  2>&1 | tee results/before-valgrind.txt
```

检测结果如图 1 所示。



```
(base) whisper@whisper:~/secp256k1-ctime-lab$ echo "Valgrind exit code: ${PIPESTATUS[0]}"  
Valgrind exit code: 99  
(base) whisper@whisper:~/secp256k1-ctime-lab$ grep -E \  
"Conditional jump|uninitialised value|ERROR SUMMARY" \  
results/before-valgrind.txt  
==16431== Conditional jump or move depends on uninitialised value(s)  
==16431== Conditional jump or move depends on uninitialised value(s)  
==16431== Conditional jump or move depends on uninitialised value(s)  
==16431== Conditional jump or move depends on uninitialised value(s)  
==16431== Conditional jump or move depends on uninitialised value(s)  
==16431== Conditional jump or move depends on uninitialised value(s)  
==16431== Conditional jump or move depends on uninitialised value(s)  
==16431== Conditional jump or move depends on uninitialised value(s)  
==16431== Conditional jump or move depends on uninitialised value(s)  
==16431== Conditional jump or move depends on uninitialised value(s)  
==16431== ERROR SUMMARY: 6606 errors from 10 contexts (suppressed: 0 from 0)  
(base) whisper@whisper:~/secp256k1-ctime-lab$
```

图 1: 修复前版本的 Valgrind 常数时间检测结果

修复前版本出现了多条：

```
Conditional jump or move depends on uninitialised value(s)
```

最终错误摘要为：

```
ERROR SUMMARY: 6606 errors from 10 contexts
```

其中，--error-exitcode=99 表示当 Valgrind 检测到错误时，以状态码 99 退出。因此，退出码 99 并不是测试程序异常崩溃，而是说明 Valgrind 成功发现了常数时间违例。

“10 contexts”表示 10 种不同的错误调用路径或错误位置；“6606 errors”表示这些上下文在测试执行过程中被动态触发的总次数。

4.3.5 分析受影响的密码操作

根据 Valgrind 调用栈，该问题影响了多条使用秘密标量的密码运算路径。

公钥生成路径

公钥生成对应的调用路径为：

```
secp256k1_fe_storage_cmov  
-> secp256k1_ge_storage_cmov  
-> secp256k1_ecmult_gen  
-> secp256k1_ec_pubkey_create_helper  
-> secp256k1_ec_pubkey_create
```

该路径计算

$$P = dG,$$

其中 d 为私钥。若条件移动产生依赖 d 的控制流，则执行轨迹可能泄露与私钥标量有关的信息。

ECDSA 签名路径

ECDSA 签名对应的调用路径为：

```
secp256k1_fe_storage_cmov  
-> secp256k1_ge_storage_cmov  
-> secp256k1_ecmult_gen  
-> secp256k1_ecdsa_sig_sign  
-> secp256k1_ecdsa_sign_inner  
-> secp256k1_ecdsa_sign
```

该路径中的固定基点标量乘法用于计算

$$R = kG,$$

其中 k 为 ECDSA 签名 nonce。若攻击者获得 nonce 的部分或全部信息，可能进一步威胁私钥安全。

ECDH 路径

ECDH 对应的调用路径为：

```
secp256k1_fe_cmov  
-> secp256k1_ecmult_const  
-> secp256k1_ecdh
```

ECDH 共享点可以表示为

$$S = d_A P_B = d_A d_B G,$$

其中 d_A 是本方私钥。调试结果显示，同一条件移动缺陷在 `ecmult_const_impl.h` 的多个内联调用位置被触发。

其他受影响路径

实验还观察到以下上层调用受到影响：

- `secp256k1_keypair_create`;
- `secp256k1_schnorr_sig_sign32`;
- `secp256k1_context_randomize`。

因此，该问题并不只影响 ECDSA，而是影响多条使用私钥、nonce 或盲化因子的秘密标量运算路径。

4.3.6 分析修复补丁

完成漏洞定位后，进一步查看修复提交 4a496a3 的提交信息和代码差异：

```
git -C secp256k1 show \  
--stat \  
--format=fuller \  
4a496a3
```

该提交的标题为：

```
ct: Use volatile "trick" in all fe/scalar cmov implementations
```

提交共修改了以下 5 个文件：

- `src/field_10x26_impl.h`;
- `src/field_5x52_impl.h`;
- `src/scalar_4x64_impl.h`;
- `src/scalar_8x32_impl.h`;

- src/scalar_low_impl.h。

由于本实验运行于 x86_64 平台，因此重点分析 field_5x52_impl.h 中的有限域条件移动，以及 scalar_4x64_impl.h 中的标量条件移动。

有限域条件移动修复

有限域元素用于表示椭圆曲线点坐标。在 64 位平台上，secp256k1_fe_cmov 使用多个 64 位分量保存有限域元素，并通过位掩码在原值 r 和候选值 a 之间进行条件选择。

field_5x52_impl.h 中的修复差异如下：

```
1 static SECP256K1_INLINE void secp256k1_fe_cmov(  
2     secp256k1_fe *r,  
3     const secp256k1_fe *a,  
4     int flag  
5 ) {  
6     uint64_t mask0, mask1;  
7 + volatile int vflag = flag;  
8     SECP256K1_CHECKMEM_CHECK_VERIFY(r->n, sizeof(r->n));  
9 - mask0 = flag + ~((uint64_t)0);  
10 + mask0 = vflag + ~((uint64_t)0);  
11     mask1 = ~mask0;  
12     ...  
13 }
```

同样的修改还应用于 secp256k1_fe_storage_cmov。该函数负责在椭圆曲线预计算表的存储形式之间进行条件选择，也是修复前 Valgrind 调用栈中直接报告秘密相关条件跳转的位置。

标量条件移动修复

标量用于表示私钥、签名 nonce 以及椭圆曲线标量乘法中的整数。对于 x86_64 平台，标量由 scalar_4x64_impl.h 中的 4 个 64 位分量表示。

该文件中的修复差异如下：

```
1 static SECP256K1_INLINE void secp256k1_scalar_cmov(  
2     secp256k1_scalar *r,  
3     const secp256k1_scalar *a,  
4     int flag  
5 ) {  
6     uint64_t mask0, mask1;  
7 + volatile int vflag = flag;  
8     SECP256K1_CHECKMEM_CHECK_VERIFY(r->d, sizeof(r->d));
```

```

9 - mask0 = flag + ~((uint64_t)0);
10 + mask0 = vflag + ~((uint64_t)0);
11   mask1 = ~mask0;
12   ...
13 }

```

有限域条件移动和标量条件移动采用了相同的修复方式：先将输入条件 `flag` 复制到易变变量 `vflag`，再由 `vflag` 构造位掩码。

volatile 修复的作用

修复前，掩码的计算方式为：

```

1 mask0 = flag + ~((uint64_t)0);
2 mask1 = ~mask0;

```

设机器字长为 w ，且

$$\text{flag} \in \{0, 1\}.$$

由于

$$\sim 0 = 2^w - 1 \equiv -1 \pmod{2^w},$$

因此

$$\text{mask0} = \text{flag} - 1 \pmod{2^w}.$$

当 $\text{flag} = 0$ 时，

$$\text{mask0} = 2^w - 1, \quad \text{mask1} = 0,$$

从而保留原值 r ；当 $\text{flag} = 1$ 时，

$$\text{mask0} = 0, \quad \text{mask1} = 2^w - 1,$$

从而选择候选值 a 。因此，对每个机器字分量都有

$$r'_i = (r_i \wedge \text{mask0}) \vee (a_i \wedge \text{mask1}),$$

其数学功能为

$$r' = \begin{cases} r, & \text{flag} = 0, \\ a, & \text{flag} = 1. \end{cases}$$

从 C 源代码看，该实现没有显式的 `if` 分支。然而，Clang 15 能够识别这段位运算的高层条件选择语义，并可能将其重新优化成依赖 `flag` 的条件跳转或条件内存访问。

修复后加入：


```
1 volatile int vflag = flag;
```

并将掩码构造修改为：

```
1 mask0 = vflag + ~((uint64_t)0);
```

该修改没有改变条件移动的数学结果。其作用是要求编译器按照易变对象的访问语义读取 `vflag`，从而抑制 Clang 15 对整个掩码表达式进行语义恢复，避免其重新生成秘密相关条件分支。

修复方法的局限性

提交说明将该方法称为一种 quick fix，即针对已经观察到的 Clang 15 编译行为实施的快速工程修复。它并不构成对所有编译器、所有处理器架构和所有未来优化规则的形式化常数时间保证。

因此，除了当前补丁外，密码库仍需要：

- 在持续集成环境中测试新的编译器版本；
- 使用 Valgrind、MemorySanitizer 等工具持续检测秘密相关控制流；
- 检查不同优化等级和不同平台生成的机器代码；
- 必要时采用编译器屏障、专用汇编或更强的常数时间验证方法。

综合来看，该补丁保持了条件移动原有的数学功能，同时阻止了本实验环境下 Clang 15 将无分支位掩码代码重新编译成秘密相关控制流。

4.3.7 构建修复后版本

修复后版本使用与修复前完全相同的编译器、优化等级和构建选项：

```
cmake \  
  -S secp-after \  
  -B build-after \  
  -DCMAKE_C_COMPILER=clang-15 \  
  -DSECP256K1_BUILD_CTIME_TESTS=ON \  
  -DSECP256K1_VALGRIND=ON \  
  -DCMAKE_BUILD_TYPE=None \  
  -DCMAKE_C_FLAGS="-O1"  
  
cmake --build build-after -j"${nproc}"
```

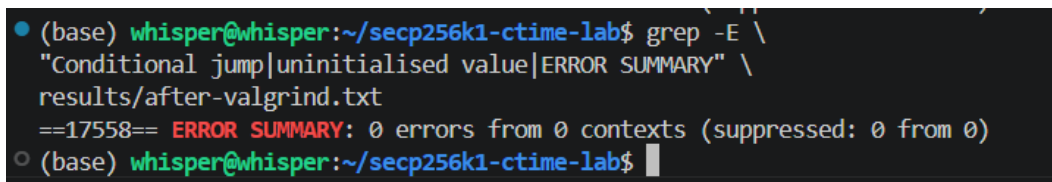
保持构建条件一致，可以避免由于编译器、优化等级或模块配置不同而造成的实验偏差。

4.3.8 执行修复后常数时间检测

修复后版本使用与修复前相同的命令执行检测：

```
valgrind \  
  --tool=memcheck \  
  --error-exitcode=99 \  
  --track-origins=yes \  
  --num-callers=20 \  
  ./build-after/src/ctime_tests \  
  2>&1 | tee results/after-valgrind.txt
```

检测结果如图 2 所示。



```
(base) whisper@whisper:~/secp256k1-ctime-lab$ grep -E \  
"Conditional jump|uninitialised value|ERROR SUMMARY" \  
results/after-valgrind.txt \  
==17558== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0) \  
(base) whisper@whisper:~/secp256k1-ctime-lab$
```

图 2: 修复后版本的 Valgrind 常数时间检测结果

修复后错误摘要为：

```
ERROR SUMMARY: 0 errors from 0 contexts
```

相同环境下不再出现

```
Conditional jump or move depends on uninitialised value(s)
```

说明应用补丁后，测试程序标记的秘密数据没有再传播到 Valgrind 能够检测到的条件控制流。

4.3.9 执行普通功能测试

为了确认补丁没有破坏密码算法的功能正确性，分别执行修复前后的普通测试程序：

```
./build-before/src/tests \  
  2>&1 | tee results/before-functional-tests.txt \  
  
./build-after/src/tests \  
  2>&1 | tee results/after-functional-tests.txt
```

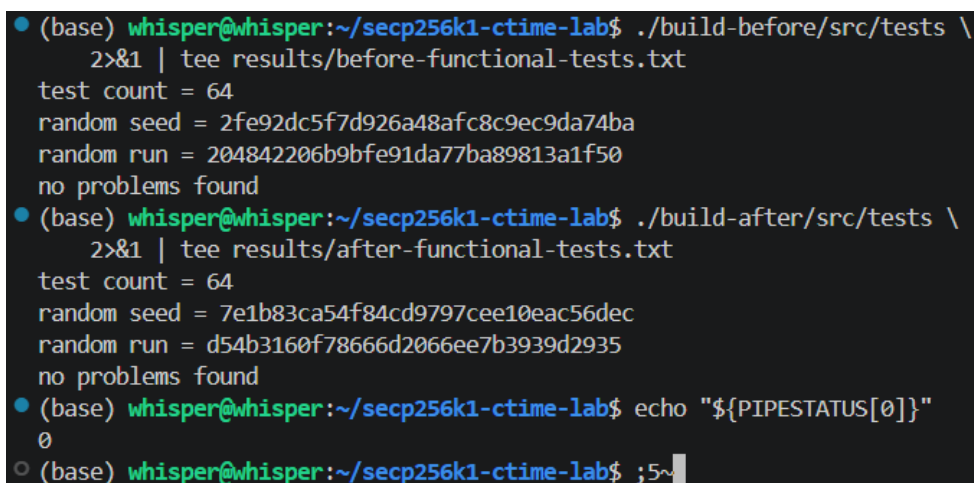
修复前测试结果为：

```
test count = 64
random seed = 2fe92dc5f7d926a48afc8c9ec9da74ba
random run = 204842206b9bfe91da77ba89813a1f50
no problems found
```

修复后测试结果为:

```
test count = 64
random seed = 7e1b83ca54f84cd9797cee10eac56dec
random run = d54b3160f78666d2066ee7b3939d2935
no problems found
```

实验结果如图 3 所示。



```
(base) whisper@whisper:~/secp256k1-ctime-lab$ ./build-before/src/tests \
2>&1 | tee results/before-functional-tests.txt
test count = 64
random seed = 2fe92dc5f7d926a48afc8c9ec9da74ba
random run = 204842206b9bfe91da77ba89813a1f50
no problems found
(base) whisper@whisper:~/secp256k1-ctime-lab$ ./build-after/src/tests \
2>&1 | tee results/after-functional-tests.txt
test count = 64
random seed = 7e1b83ca54f84cd9797cee10eac56dec
random run = d54b3160f78666d2066ee7b3939d2935
no problems found
(base) whisper@whisper:~/secp256k1-ctime-lab$ echo "${PIPESTATUS[0]}"
0
(base) whisper@whisper:~/secp256k1-ctime-lab$ ;5~
```

图 3: 修复前后普通功能测试结果

两组普通功能测试均通过。随机种子不同是测试程序正常生成随机测试数据的结果,不影响实验结论。

该结果说明,修复前版本虽然存在秘密相关控制流,但其密码运算输出仍然正确。该问题属于实现层侧信道缺陷,而不是签名、公钥生成或 ECDH 计算结果错误。

4.3.10 修复前后结果汇总

修复前后实验结果如表 5 所示。

表 5: 修复前后实验结果对比

对比项	修复前	修复后
源代码版本	464a9115...	4a496a36...
编译器	Clang 15.0.7	Clang 15.0.7
优化等级	-O1	-O1
ctime_tests	开启	开启
Valgrind 秘密数据跟踪	开启	开启
条件跳转错误上下文	10	0
动态错误触发次数	6606	0
普通功能测试	通过	通过
内存泄漏	未发现	未发现

由表 5 可知, 修复前后版本均能通过普通功能测试, 说明补丁没有改变密码算法的计算结果。在相同 Clang 15.0.7、-O1 和 Valgrind 配置下, 常数时间错误由 6606 次降为 0, 表明提交 4a496a3 有效消除了本实验环境中检测到的秘密相关条件控制流。

4.4 结果讨论与安全影响

4.4.1 功能正确性与实现安全性的区别

本实验中, 修复前版本的普通功能测试显示 no problems found, 说明密钥生成、签名和其他密码运算的输出仍然符合预期。然而, Valgrind 同时检测出秘密相关条件跳转。这表明:

输出正确 $\nRightarrow$ 密码实现安全.

普通测试主要验证

$$f(x) = y,$$

而常数时间安全测试还要验证不同秘密下的执行轨迹是否保持秘密无关。

4.4.2 与 Bitcoin 安全的关系

Bitcoin 的交易授权最终依赖私钥控制。即使 ECDSA 或 Schnorr 的数学安全性没有被直接破坏, 若底层实现通过时间、缓存或分支预测泄露私钥或 nonce 信息, 系统整体安全性仍可能下降。

因此, 密码库安全不仅包括算法公式正确, 还包括:

- 秘密相关操作保持常数时间;

- 随机数和 nonce 安全生成;
- 有限域与标量运算正确归约;
- 编译器优化后仍保持安全属性;
- 在多种平台和工具链上持续验证。

5 固定基点标量乘法性能优化

前文已经说明, 公钥生成和 ECDSA 签名分别需要计算 dG 和 kG , 并通过固定基点标量乘法函数 `ecmult_gen` 完成。本节进一步分析该函数在不同版本中的算法更新, 并通过基准测试评价其性能和空间开销。

5.1 优化对象与版本变化

`libsecp256k1` 的变更记录显示, `v0.5.0` 更换了签名和公钥生成所使用的点乘算法, 并明确指出该修改提高了这两类操作的性能。

旧算法支持的预计算表大小为:

32 KiB, 64 KiB, 512 KiB.

`v0.5.0` 新算法支持的预计算表大小为:

2 KiB, 22 KiB, 86 KiB.

`v0.5.1` 又将默认预计算表大小由 22 KiB 调整为 86 KiB。相关版本更新如图 4 所示。

```
7
8  ## [0.5.1] - 2024-08-01
9
10 ##### Added
11 - Added usage example for an ElligatorSwift key exchange.
12
13 ##### Changed
14 - The default size of the precomputed table for signing was changed from 22 KiB to 86 KiB. The size can be changed with the configure option '--ecmult-gen-kb' ('SECP256K1_ECMULT_GEN_KB' for CMake).
15 - 'auto' is no longer an accepted value for the '--with-ecmult-window' and '--with-ecmult-gen-kb' configure options (this also applies to 'SECP256K1_ECMULT_WINDOW_SIZE' and 'SECP256K1_ECMULT_GEN_KB' in CMake). To achieve the same configuration as previously provided by the "auto" value, omit setting the configure option explicitly.
16
17 ##### Fixed
18 - Fixed compilation when the extrakeys module is disabled.
19
20 ##### ABI Compatibility
21 The ABI is backward compatible with versions 0.5.0, 0.4.x and 0.3.x.
22
23 ## [0.5.0] - 2024-05-06
24
25 ##### Added
26 - New function 'secp256k1_ec_pubkey_sort' that sorts public keys using lexicographic (of compressed serialization) order.
27
28 ##### Changed
29 - The implementation of the point multiplication algorithm used for signing and public key generation was changed, resulting in improved performance for those operations.
30 - The related configure option '--ecmult-gen-precision' was replaced with '--ecmult-gen-kb' ('SECP256K1_ECMULT_GEN_KB' for CMake).
31 - This changes the supported precomputed table sizes for these operations. The new supported sizes are 2 KiB, 22 KiB, or 86 KiB (while the old supported sizes were 32 KiB, 64 KiB, or 512 KiB).
```

图 4: `v0.5.0` 和 `v0.5.1` 中与固定基点标量乘法有关的版本更新

该变化说明，v0.5.0 的优化目标并非简单增大预计算表，而是重新设计固定基点标量乘法算法，以改善运行时间和存储空间之间的权衡。

5.2 新旧固定基点标量乘法算法

5.2.1 v0.4.1 旧固定宽度预计算算法

v0.4.1 根据参数 `PREC_BITS` 将 256 位标量划分为固定宽度的分组。设每组包含 b 位，则分组数量为：

$$N = \frac{256}{b}.$$

每组可能具有 2^b 种取值。由于分组值来自私钥或签名 `nonce`，程序不能直接将其作为数组索引，而需要遍历该组的全部候选表项，再通过常数时间条件移动选出目标点。

因此，旧算法的点加次数为：

$$N_{\text{add}} = \frac{256}{b},$$

条件移动次数为：

$$N_{\text{cmov}} = \frac{256}{b} \cdot 2^b.$$

三种旧配置的理论开销如表 6 所示。

表 6: v0.4.1 旧固定宽度预计算算法的理论开销

<code>PREC_BITS</code>	表大小	分组数	条件移动次数	点加次数
2	32 KiB	128	512	128
4	64 KiB	64	1024	64
8	512 KiB	32	8192	32

随着 b 增大，点加次数下降，但每组需要扫描的候选点数量按指数增长。因此，旧算法的性能不一定随预计算表增大而单调提高。

5.2.2 v0.5.0 signed-digit multi-comb 算法

v0.5.0 引入带符号数字多梳状算法，即 `signed-digit multi-comb`。其主要参数包括：

- `COMB_BLOCKS`：标量被划分的块数，记为 B ；
- `COMB_TEETH`：每张表同时覆盖的比特数，记为 T ；
- `COMB_SPACING`：梳齿之间的距离，记为 S 。

构建系统通过参数 SECP256K1_ECMULT_GEN_KB 选择不同的 multi-comb 配置。2 KiB、22 KiB 和 86 KiB 分别对应：

$$(B, T) = (2, 5), \quad (11, 6), \quad (43, 6).$$

其源码配置如图 5 所示。

```

92
93 set(SECP256K1_ECMULT_GEN_KB 86 CACHE STRING "The size of the precomputed table for signing in multiples of 1024
bytes (on typical platforms). Larger values result in possibly better signing or key generation performance at the cost
of a larger table. Valid choices are 2, 22, 86. The default value is a reasonable setting for desktop machines (currentl
y 86). [default=86]")
94 set_property(CACHE SECP256K1_ECMULT_GEN_KB PROPERTY STRINGS 2 22 86)
95 check_string_option_value(SECP256K1_ECMULT_GEN_KB)
96 if(SECP256K1_ECMULT_GEN_KB EQUAL 2)
97   add_compile_definitions(COMB_BLOCKS=2)
98   add_compile_definitions(COMB_TEETH=5)
99 elseif(SECP256K1_ECMULT_GEN_KB EQUAL 22)
100   add_compile_definitions(COMB_BLOCKS=11)
101   add_compile_definitions(COMB_TEETH=6)
102 elseif(SECP256K1_ECMULT_GEN_KB EQUAL 86)
103   add_compile_definitions(COMB_BLOCKS=43)
104   add_compile_definitions(COMB_TEETH=6)
105 endif()

```

图 5: 预计算表大小与 signed-digit multi-comb 参数的对应关系

梳齿间距定义为：

$$S = \left\lceil \frac{256}{BT} \right\rceil.$$

预计算表项数为：

$$N_{\text{table}} = B \cdot 2^{T-1}.$$

运行时点加次数为：

$$N_{\text{add}} = B \cdot S,$$

倍点次数为：

$$N_{\text{double}} = S - 1.$$

源码中给出的参数和运算量关系如图 6 所示。

```

14 /* Configuration parameters for the signed-digit multi-comb algorithm:
15 *
16 * - COMB_BLOCKS is the number of blocks the input is split into. Each
17 *   has a corresponding table.
18 * - COMB_TEETH is the number of bits simultaneously covered by one table.
19 * - COMB_RANGE is the number of bits in supported scalars. For production
20 *   purposes, only 256 is reasonable, but smaller numbers are supported for
21 *   exhaustive test mode.
22 *
23 * The comb's spacing (COMB_SPACING), or the distance between the teeth,
24 * is defined as ceil(COMB_RANGE / (COMB_BLOCKS * COMB_TEETH)). Each block covers
25 * COMB_SPACING * COMB_TEETH consecutive bits in the input.
26 *
27 * The size of the precomputed table is COMB_BLOCKS * (1 <= (COMB_TEETH - 1))
28 * secp256k1_ge_storages.
29 *
30 * The number of point additions equals COMB_BLOCKS * COMB_SPACING. Each point
31 * addition involves a cmov from (1 <= (COMB_TEETH - 1)) table entries and a
32 * conditional negation.
33 *
34 * The number of point doublings is COMB_SPACING - 1. */
35
36 #if defined(EXHAUSTIVE_TEST_ORDER)
37 /* We need to control these values for exhaustive tests because
38 * the table cannot have infinities in them (secp256k1_ge_storage
(.venv) wen@localhost:~/secp256k1-exp/secp256k1$

```

图 6: signed-digit multi-comb 算法的配置参数及其计算与存储开销

三种新配置的理论运算量如表 7 所示。

表 7: signed-digit multi-comb 算法不同配置的理论开销

表大小	B	T	S	表项数	点加次数	倍点次数
2 KiB	2	5	26	32	52	25
22 KiB	11	6	4	352	44	3
86 KiB	43	6	1	1376	43	0

从 2 KiB 增大到 22 KiB 时，点加次数由 52 次降至 44 次，倍点次数由 25 次降至 3 次，因此预计会获得较明显的性能提升。从 22 KiB 增大到 86 KiB 时，仅进一步减少 1 次点加和 3 次倍点，因此性能仍可提高，但边际收益会减小。

5.2.3 带符号表示与半表存储

signed-digit multi-comb 使用带符号数字表示。对于比特 s_i ，有：

$$2s_i - 1 \in \{-1, +1\}.$$

因此，每个比特表示正点或负点，而不是简单的“加点或不加点”。算法利用以下对称关系：

$$\text{table}(b, m \oplus \text{mask}(b)) = -\text{table}(b, m).$$

若椭圆曲线点为：

$$P = (x, y),$$

则其负点为:

$$-P = (x, -y \bmod p).$$

因此, 原本需要保存的 2^T 个组合点可以缩减为:

$$2^{T-1}.$$

对应源码说明如图 7 所示。

```
(.venv) wen@localhost:~/secp256k1-exp/secp256k1$ nl -ba src/ecmult_gen_impl.h | sed -n '176,189p'
176 * The last question is how to implement the table(b, m) function. For any value of b,
177 * m=(d & mask(b)) can only take on at most 2^COMB_TEETH possible values (the last one may have
178 * fewer as there mask(b) may exceed the curve order). So we could create COMB_BLOCK tables
179 * which contain a value for each such m value.
180 *
181 * Now note that if m=(d & mask(b)), then flipping the relevant bits of m results in negating
182 * the result of table(b, m). This is because table(b, m XOR mask(b)) = table(b, mask(b) - m) =
183 * (mask(b) - m - mask(b)/2)*G = (-m + mask(b)/2)*G = -(m - mask(b)/2)*G = -table(b, m).
184 * Because of this it suffices to only store the first half of the m values for every b. If an
185 * entry from the second half is needed, we look up its bit-flipped version instead, and negate
186 * it.
187 *
188 * secp256k1_ecmult_gen_prec_table[b][index] stores the table(b, m) entries. Index
189 * is the relevant mask(b) bits of m packed together without gaps. */
```

图 7: 利用椭圆曲线点的正负对称性将预计算表规模减半

新算法仍然使用条件移动选择表项, 不直接使用秘密索引查表。因此, 其性能优化是在保持秘密无关控制流和统一内存访问设计的前提下完成的。

5.3 性能实验设计

5.3.1 实验环境与性能指标

性能实验运行于 WSL2 Linux 环境, 处理器为 Intel Core i5-13500H, 编译器为 GCC 13.3.0, 使用 CMake 3.28.3 和 Ninja 1.11.1。所有版本均采用 Release 模式构建静态库, 并关闭可选汇编实现, 以减少其他实现差异对固定基点标量乘法实验的干扰。

测试进程固定在逻辑 CPU 0 上, 每项 benchmark 的内部迭代次数为 50000。测量的密码操作包括:

- ECDSA 签名: `ecdsa_sign`;
- 公钥生成: `ec_keygen`;
- ECDSA 验证: `ecdsa_verify`。

ECDSA 签名和公钥生成调用 `ecmult_gen`, 而 ECDSA 验证使用不同的双标量乘法路径, 因此将验证操作作为负对照。

设旧配置的平均耗时为 T_{old} ，新配置的平均耗时为 T_{new} ，则时延降低比例定义为：

$$\text{Improvement} = \frac{T_{\text{old}} - T_{\text{new}}}{T_{\text{old}}} \times 100\%.$$

5.3.2 v0.5.1 不同预计算表大小实验

第一组实验固定版本为 v0.5.1，仅改变参数：

$$\text{SECP256K1_ECMULT_GEN_KB} \in \{2, 22, 86\}.$$

该实验用于分析在算法保持不变的情况下，预计算表大小对签名、公钥生成、验证和编译产物大小的影响。

5.3.3 v0.4.1 与 v0.5.0 跨版本实验

第二组实验比较 v0.4.1 的旧固定宽度预计算算法和 v0.5.0 的 signed-digit multi-comb 算法，共包含六种配置，如表 8 所示。

表 8: 固定基点标量乘法跨版本实验分组

配置	版本	算法	构建参数	名义表大小
Old 32	v0.4.1	旧固定宽度预计算	PREC_BITS=2	32 KiB
Old 64	v0.4.1	旧固定宽度预计算	PREC_BITS=4	64 KiB
Old 512	v0.4.1	旧固定宽度预计算	PREC_BITS=8	512 KiB
New 2	v0.5.0	signed-digit multi-comb	ECMULT_GEN_KB=2	2 KiB
New 22	v0.5.0	signed-digit multi-comb	ECMULT_GEN_KB=22	22 KiB
New 86	v0.5.0	signed-digit multi-comb	ECMULT_GEN_KB=86	86 KiB

构建完成后，分别核对旧算法的 `ecmult gen precision bits` 和新算法的 `ecmult gen table size`，同时确认 `benchmark` 已开启、汇编实现已关闭且构建类型均为 `Release`。构建配置核验结果如图 8 所示。

```

===== v0.4.1 PREC_BITS=2 =====
-- The C compiler identification is GNU 13.3.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
ecmult gen precision bits ..... 2
assembly ..... OFF
benchmark ..... ON
C compiler ..... /usr/bin/cc
- CMAKE_BUILD_TYPE ..... Release

===== v0.4.1 PREC_BITS=4 =====
-- The C compiler identification is GNU 13.3.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
ecmult gen precision bits ..... 4
assembly ..... OFF
benchmark ..... ON
C compiler ..... /usr/bin/cc
- CMAKE_BUILD_TYPE ..... Release

===== v0.4.1 PREC_BITS=8 =====
-- The C compiler identification is GNU 13.3.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
ecmult gen precision bits ..... 8
assembly ..... OFF
benchmark ..... ON
C compiler ..... /usr/bin/cc
- CMAKE_BUILD_TYPE ..... Release

```

(a) v0.4.1 旧算法的三种配置

```

===== v0.5.0 2 KiB =====
-- The C compiler identification is GNU 13.3.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
ecmult gen table size ..... 2 KiB
assembly ..... OFF
benchmark ..... ON
C compiler ..... /usr/bin/cc
- CMAKE_BUILD_TYPE ..... Release

===== v0.5.0 22 KiB =====
-- The C compiler identification is GNU 13.3.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
ecmult gen table size ..... 22 KiB
assembly ..... OFF
benchmark ..... ON
C compiler ..... /usr/bin/cc
- CMAKE_BUILD_TYPE ..... Release

===== v0.5.0 86 KiB =====
-- The C compiler identification is GNU 13.3.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
ecmult gen table size ..... 86 KiB
assembly ..... OFF
benchmark ..... ON
C compiler ..... /usr/bin/cc
- CMAKE_BUILD_TYPE ..... Release

```

(b) v0.5.0 新算法的三种配置

图 8: 固定基点标量乘法跨版本实验的六种构建配置核验

5.3.4 重复测量与统计方法

为降低 CPU 温度、睿频变化、缓存状态和固定运行顺序造成的偏差，跨版本实验采用 12 轮交错重复。六种配置在每轮中按照不同顺序执行，使每种配置在第一至第六个运行位置中各出现两次。

所有 benchmark 均使用以下形式执行：

```
SECP256K1_BENCH_ITERS=50000 taskset -c 0 ./bench \
    ecdsa_sign ec_keygen ecdsa_verify
```

跨版本实验共生成：

$$6 \times 12 = 72$$

个结果文件。每个文件包含 3 项密码操作，因此共得到：

$$72 \times 3 = 216$$

条操作级记录。

对于每种配置，计算 12 次外部重复的均值、中位数、样本标准差和 95% 置信区间。所有正式结果均保留，未删除单次波动数据。

5.4 实验结果

5.4.1 v0.5.1 不同预计算表大小的结果

v0.5.1 内部参数实验结果如表 9 所示。

表 9: v0.5.1 不同预计算表大小下的平均耗时

预计算表大小	ECDSA 签名 (μs)	公钥生成 (μs)	ECDSA 验证 (μs)
2 KiB	19.73	14.39	28.80
22 KiB	17.88	12.38	28.70
86 KiB	17.39	12.00	28.70

与 2 KiB 配置相比, 86 KiB 配置使 ECDSA 签名时延降低 11.87%, 公钥生成时延降低 16.62%, 而 ECDSA 验证仅变化约 0.35%。从 2 KiB 增大到 22 KiB 是主要性能收益阶段, 继续增大到 86 KiB 后仍有提升, 但边际收益明显下降。

5.4.2 跨版本平均性能结果

六种配置的正式测试结果如表 10 所示。

表 10: v0.4.1 与 v0.5.0 跨版本性能测试结果

配置	表大小	ECDSA 签名 (μs)	公钥生成 (μs)	ECDSA 验证 (μs)
Old 32	32 KiB	29.04	23.74	28.88
Old 64	64 KiB	19.72	14.27	28.69
Old 512	512 KiB	29.63	23.94	28.51
New 2	2 KiB	19.76	14.33	28.79
New 22	22 KiB	17.93	12.54	28.76
New 86	86 KiB	17.38	11.96	28.74

旧算法的 64 KiB 配置明显优于 32 KiB 和 512 KiB 配置, 说明其性能并不随预计算表大小单调提高。新算法的 22 KiB 和 86 KiB 配置则取得了最低的签名和公钥生成时延。

5.4.3 性能提升百分比

按照低配置、默认配置和高配置三个层级比较新旧算法, 结果如表 11 所示。

表 11: v0.5.0 新算法相对 v0.4.1 旧算法的性能提升

比较	ECDSA 签名提升	公钥生成提升	名义表大小缩减
Old 32 → New 2	31.96%	39.66%	93.75%
Old 64 → New 22	9.04%	12.08%	65.63%
Old 512 → New 86	41.37%	50.05%	83.20%

三组配置的性能提升百分比如图 9 所示。

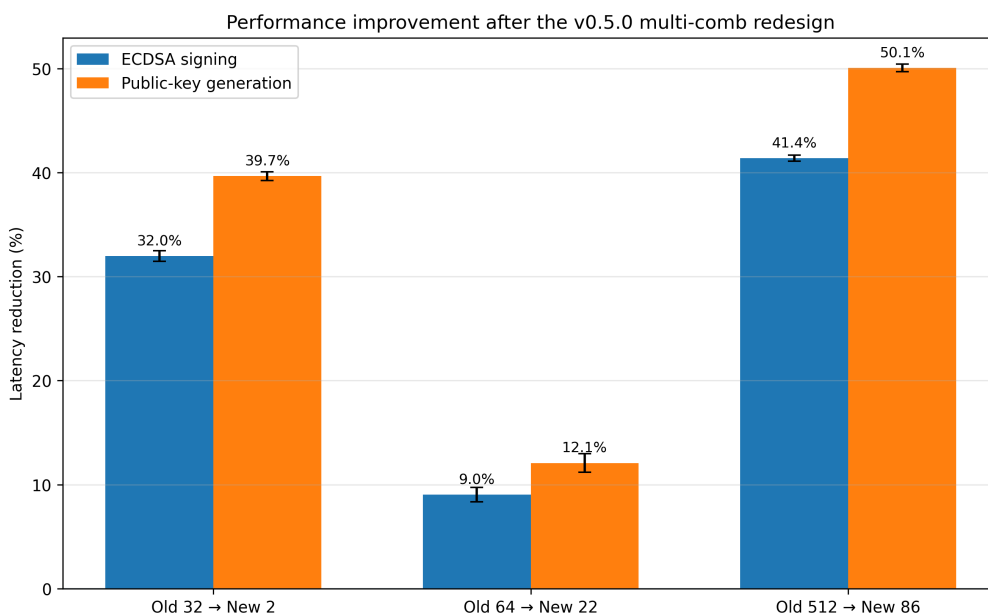


图 9: v0.5.0 signed-digit multi-comb 算法相对旧算法的性能提升

其中，最具有代表性的默认配置比较为：

Old 64 KiB → New 22 KiB.

新算法在将名义预计算表由 64 KiB 缩小至 22 KiB 的同时，使 ECDSA 签名和公钥生成时延分别降低 9.04% 和 12.08%。

5.4.4 编译产物大小

六种配置对应的静态库大小如表 12 所示。

表 12: 不同固定基点标量乘法配置下的静态库大小

配置	名义预计算表大小	静态库大小 (KiB)
Old 32	32 KiB	1163.38
Old 64	64 KiB	1195.41
Old 512	512 KiB	1643.41
New 2	2 KiB	1134.37
New 22	22 KiB	1154.34
New 86	86 KiB	1219.00

默认配置由 Old 64 调整为 New 22 后, 名义预计算表缩小 65.63%, 静态库大小减少约 41.07 KiB, 即 3.44%。高配置由 Old 512 调整为 New 86 后, 静态库大小减少约 424.41 KiB, 即 25.82%。

5.4.5 时间-空间关系

两代算法在不同离散预计算表配置下的 ECDSA 签名和公钥生成时延分别如图 10 和图 11 所示。每个数据点对应一种实际支持的构建配置, 误差棒表示 95% 置信区间, 点与点之间不表示其他表大小下的连续插值结果。

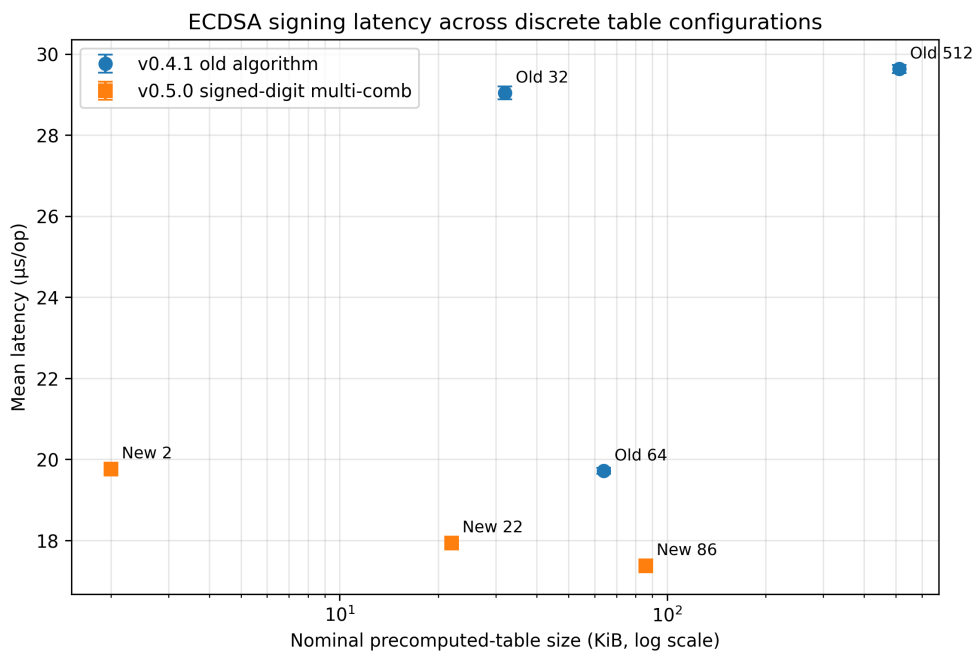


图 10: 不同离散预计算表配置下的 ECDSA 签名时延

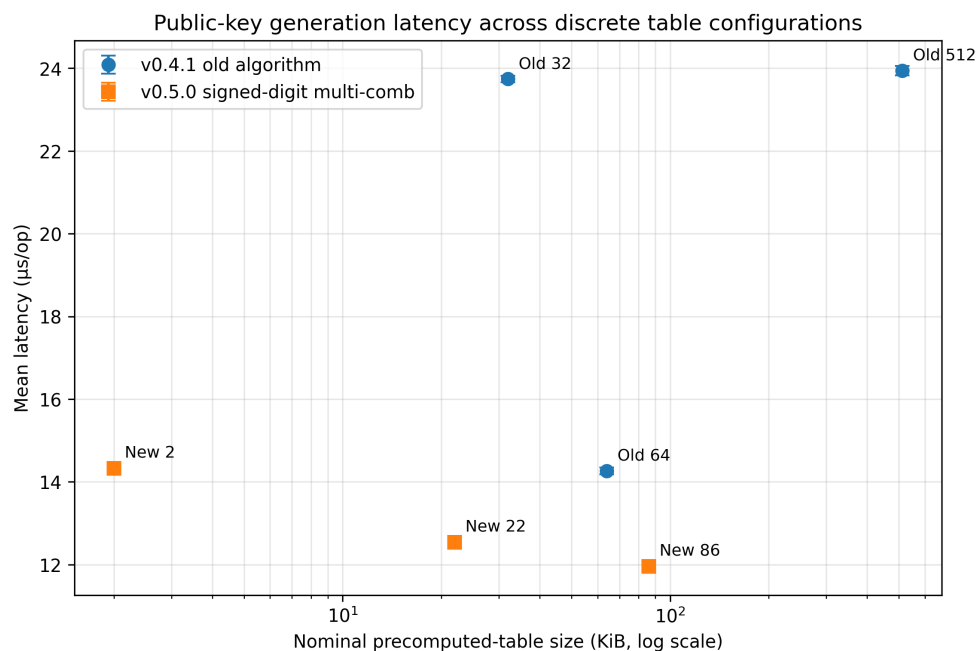


图 11: 不同离散预计算表配置下的公钥生成时延

旧算法的性能随表大小呈现明显的非单调变化，而新算法在更小的表空间下取得了更低时延，说明 v0.5.0 的算法更新改善了性能-空间前沿。

5.4.6 ECDSA 验证负对照

六种配置下 ECDSA 验证的平均耗时均处于：

$$28.5 \sim 28.9 \mu s$$

范围内，如图 12 所示。

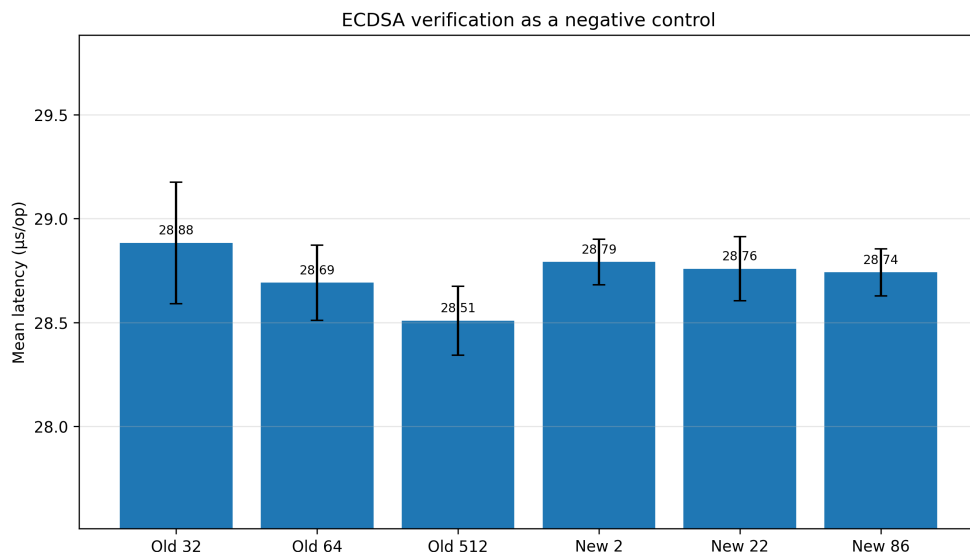


图 12: 六种配置下的 ECDSA 验证平均耗时

ECDSA 验证没有随 `ecmult_gen` 的表大小变化出现系统性加速，说明签名和公钥生成的性能提升具有明确的代码路径针对性。

5.5 结果分析与讨论

5.5.1 新算法改善了性能-空间权衡

New 2 配置仅使用 2 KiB 预计算表，其 ECDSA 签名和公钥生成耗时分别为 19.76 μs 和 14.33 μs ，与 Old 64 的 19.72 μs 和 14.27 μs 基本相同。

New 2 的名义预计算表仅为 Old 64 的：

$$\frac{2}{64} = \frac{1}{32},$$

而两项操作的性能差异均不足 0.5%。这表明新算法显著提高了单位预计算空间所能获得的性能。

5.5.2 旧算法非单调性能的原因

旧算法中，32 KiB 配置需要执行 128 次点加，大量椭圆曲线点运算造成较高开销。

512 KiB 配置虽然将点加次数降至 32 次，但每个分组需要扫描 256 个候选点，总条件移动次数达到：

$$32 \times 256 = 8192.$$

此外，512 KiB 预计算表超过单个核心的 L1 数据缓存容量，大量常数时间表扫描会增加缓存和内存访问压力。因此，旧算法的 64 KiB 配置在点加次数和表扫描次数之间取得了更合理的平衡。

5.5.3 新算法性能提升的数学原因

multi-comb 算法将固定基点标量乘法组织为 COMB_SPACING 轮计算。每一轮首先进行必要的倍点，随后遍历全部 COMB_BLOCKS，从对应的预计算表中选择一个点并累加。其核心实现可概括为以下代码。为突出算法结构，代码中省略了标量位提取等细节。

```
1 for (i = 0; i < COMB_SPACING; ++i) {
2     if (i != 0) {
3         secp256k1_gej_double(r, r);
4     }
5
6     for (block = 0; block < COMB_BLOCKS; ++block) {
7         /* Extract and recode the bits for this block. */
8
9         for (index = 0; index < COMB_POINTS; ++index) {
10             secp256k1_ge_storage_cmov(
11                 &adds,
12                 &secp256k1_ecmult_gen_prec_table[block][index],
13                 index == abs
14             );
15         }
16
17         secp256k1_ge_from_storage(&add, &adds);
18         secp256k1_fe_negate(&neg, &add.y, 1);
19         secp256k1_fe_cmov(&add.y, &neg, sign);
20         secp256k1_gej_add_ge(r, r, &add);
21     }
22 }
```

代码中的外层循环执行 $S = \text{COMB_SPACING}$ 轮。第一轮不需要倍点，之后每轮执行一次 `secp256k1_gej_double`，因此倍点次数为：

$$N_{\text{double}} = S - 1.$$

每轮均遍历 $B = \text{COMB_BLOCKS}$ 个块，并对每个块执行一次 `secp256k1_gej_add_ge`，因此点加次数为：

$$N_{\text{add}} = B \cdot S.$$

对于每次点加，程序还要遍历 $\text{COMB_POINTS} = 2^{T-1}$ 个候选表项，

并使用 `secp256k1_ge_storage_cmov` 进行常数时间选择。随后，`secp256k1_fe_cmov` 根据带符号数字决定是否将点的 y 坐标取负。该过程实现了正点与负点的统一选择，同

时避免将秘密窗口值直接作为数组索引。

将三种配置的参数代入可得：

2 KiB : 52次点加 + 25次倍点,
22 KiB : 44次点加 + 3次倍点,
86 KiB : 43次点加 + 0次倍点.

因此, 从 2 KiB 增大到 22 KiB 时, 算法减少了 8 次点加和 22 次倍点, 性能提升最明显; 从 22 KiB 增大到 86 KiB 时, 仅进一步减少 1 次点加和 3 次倍点, 因此仍能提高性能, 但边际收益明显减小。

实验中公钥生成的性能提升高于 ECDSA 签名。其原因是公钥生成的主要计算直接集中于：

$$P = dG,$$

而完整 ECDSA 签名除了计算：

$$R = kG$$

以外, 还需要执行 nonce 处理、标量乘法、模逆以及签名结果规范化等操作。因此, `ecmult_gen` 加速在公钥生成总耗时中的占比更高, 而在完整 ECDSA 签名中的收益会被其他固定开销部分稀释。

5.5.4 性能优化与常数时间安全

新算法仍使用条件移动遍历并选择预计算表项, 没有将秘密窗口值直接用作数组索引。因此, 性能提高并不是通过取消常数时间防护获得的。

安全修复案例说明, 条件移动的源码形式还可能受到编译器优化影响; 性能优化案例则进一步说明, 在保持秘密无关查表设计的前提下, 仍可通过改进标量分解和预计算表结构提高运行效率。两部分共同体现了密码库工程实现中安全性与性能之间的综合权衡。

5.5.5 实验局限性

本性能实验仅在 Intel Core i5-13500H 和 WSL2 环境中完成, 绝对耗时可能受到虚拟化调度、处理器缓存结构和编译器行为影响。此外, v0.4.1 与 v0.5.0 之间还包含其他源码变化, 跨版本性能差异不能完全归因于单一提交; 微基准测试也不能完全代表真实 Bitcoin 钱包或节点负载。尽管如此, 签名和公钥生成均表现出稳定提升, 而验证负对照基本不变, 与 `ecmult_gen` 的算法更新和代码路径一致。

6 总结

本实验首先复现了 libsecp256k1 提交 4a496a3 修复前的常数时间安全问题。在 Clang 15 和 -O1 环境下,修复前版本出现 6606 次秘密相关条件控制流错误,应用 volatile 补丁后错误降为 0,同时修复前后均通过普通功能测试。这说明该问题属于实现层侧信道风险,而不是密码运算结果错误。

性能优化实验进一步分析了 ecmult_gen 固定基点标量乘法。在默认配置比较中,v0.5.0 的 22 KiB multi-comb 配置相较于 v0.4.1 的 64 KiB 配置,使 ECDSA 签名和公钥生成性能分别提高 9.04% 和 12.08%,同时将名义预计算表缩小 65.63%。ECDSA 验证基本保持不变,与其使用不同点乘路径的源码结构一致。

两个案例共同表明,密码库的工程安全不仅取决于数学公式是否正确,还取决于编译器优化后的执行轨迹、预计算表结构、缓存行为以及常数时间约束。libsecp256k1 通过持续的安全修复和算法优化,在保护秘密数据的同时改善了比特币密钥生成与数字签名的运行效率。

A 主要实验命令

A.1 建立工作区

```
git clone https://github.com/bitcoin-core/secp256k1.git
cd secp256k1

git worktree add ../secp-before '4a496a3^'
git worktree add ../secp-after 4a496a3
```

A.2 构建修复前版本

```
cmake \
  -S secp-before \
  -B build-before \
  -DCMAKE_C_COMPILER=clang-15 \
  -DSECP256K1_BUILD_CTIME_TESTS=ON \
  -DSECP256K1_VALGRIND=ON \
  -DCMAKE_BUILD_TYPE=None \
  -DCMAKE_C_FLAGS="-O1"

cmake --build build-before -j"${nproc}"
```

A.3 构建修复后版本

```
cmake \  
  -S secp-after \  
  -B build-after \  
  -DCMAKE_C_COMPILER=clang-15 \  
  -DSECP256K1_BUILD_CTIME_TESTS=ON \  
  -DSECP256K1_VALGRIND=ON \  
  -DCMAKE_BUILD_TYPE=None \  
  -DCMAKE_C_FLAGS="-O1"  
  
cmake --build build-after -j"${nproc}"
```

A.4 执行常数时间测试

```
valgrind \  
  --tool=memcheck \  
  --error-exitcode=99 \  
  --track-origins=yes \  
  --num-callers=20 \  
  ./build-before/src/ctime_tests \  
  2>&1 | tee results/before-valgrind.txt  
  
valgrind \  
  --tool=memcheck \  
  --error-exitcode=99 \  
  --track-origins=yes \  
  --num-callers=20 \  
  ./build-after/src/ctime_tests \  
  2>&1 | tee results/after-valgrind.txt
```

A.5 执行普通功能测试

```
./build-before/src/tests \  
  2>&1 | tee results/before-functional-tests.txt  
  
./build-after/src/tests \  
  2>&1 | tee results/after-functional-tests.txt
```

A.6 查看补丁

```
git -C secp256k1 show --stat --format=fuller 4a496a3

git -C secp256k1 diff \
    '4a496a3^' \
    4a496a3 \
    -- src/field_5x52_impl.h

git -C secp256k1 diff \
    '4a496a3^' \
    4a496a3 \
    -- src/scalar_4x64_impl.h
```

A.7 建立性能实验工作区

首先克隆 libsecp256k1 仓库，并分别建立 v0.4.1、v0.5.0 和 v0.5.1 三个独立工作区：

```
mkdir -p ~/secp256k1-exp
cd ~/secp256k1-exp

git clone https://github.com/bitcoin-core/secp256k1.git
cd secp256k1
git fetch --tags

git worktree add --detach \
    ~/secp256k1-exp/secp256k1-v041 \
    v0.4.1

git worktree add --detach \
    ~/secp256k1-exp/secp256k1-v050 \
    v0.5.0

git worktree add --detach \
    ~/secp256k1-exp/secp256k1-v051 \
    v0.5.1

mkdir -p ~/secp256k1-exp/results-cross-version
```

分别确认三个工作区对应的正式版本：

```
git -C ~/secp256k1-exp/secp256k1-v041 \  
    describe --tags --exact-match  
  
git -C ~/secp256k1-exp/secp256k1-v050 \  
    describe --tags --exact-match  
  
git -C ~/secp256k1-exp/secp256k1-v051 \  
    describe --tags --exact-match
```

预期输出依次为：

```
v0.4.1  
v0.5.0  
v0.5.1
```

A.8 构建六种性能配置

跨版本实验包含 v0.4.1 旧算法的三种配置和 v0.5.0 新算法的三种配置。所有配置均采用 GCC、Release 模式和静态库构建，开启 benchmark，关闭普通测试、常数时间测试和可选汇编实现。

A.8.1 构建 v0.4.1 旧算法配置

v0.4.1 使用参数 SECP256K1_ECMULT_GEN_PREC_BITS 控制固定基点预计算精度，合法值为 2、4 和 8。

```
cd ~/secp256k1-exp/secp256k1-v041  
  
rm -rf build-old-p2 build-old-p4 build-old-p8  
  
for bits in 2 4 8  
do  
    cmake \  
        -S . \  
        -B "build-old-p${bits}" \  
        -G Ninja \  
        -DCMAKE_BUILD_TYPE=Release \  
        -DBUILD_SHARED_LIBS=OFF \  
        -DSECP256K1_ECMULT_GEN_PREC_BITS="${bits}" \  
        -DSECP256K1_ASM=OFF \  
done
```

```
-DSECP256K1_BUILD_BENCHMARK=ON \  
-DSECP256K1_BUILD_TESTS=OFF \  
-DSECP256K1_BUILD_EXHAUSTIVE_TESTS=OFF \  
-DSECP256K1_BUILD_CTIME_TESTS=OFF  
  
cmake \  
  --build "build-old-p${bits}" \  
  --parallel "$(nproc)"  
done
```

三种构建目录分别对应：

- build-old-p2: 32 KiB 预计算表；
- build-old-p4: 64 KiB 预计算表；
- build-old-p8: 512 KiB 预计算表。

A.8.2 构建 v0.5.0 新算法配置

v0.5.0 使用参数 SECP256K1_ECMULT_GEN_KB 直接指定预计算表大小。

```
cd ~/secp256k1-exp/secp256k1-v050  
  
rm -rf build-new-2k build-new-22k build-new-86k  
  
for kb in 2 22 86  
do  
  cmake \  
    -S . \  
    -B "build-new-${kb}k" \  
    -G Ninja \  
    -DCMAKE_BUILD_TYPE=Release \  
    -DBUILD_SHARED_LIBS=OFF \  
    -DSECP256K1_ECMULT_GEN_KB="${kb}" \  
    -DSECP256K1_ASM=OFF \  
    -DSECP256K1_BUILD_BENCHMARK=ON \  
    -DSECP256K1_BUILD_TESTS=OFF \  
    -DSECP256K1_BUILD_EXHAUSTIVE_TESTS=OFF \  
    -DSECP256K1_BUILD_CTIME_TESTS=OFF  
  
  cmake \  
    --build "build-new-${kb}k" \  
    --parallel "$(nproc)"  
done
```

```
--build "build-new-${kb}k" \  
--parallel "$(nproc)"  
done
```

三种构建目录分别为：

- build-new-2k;
- build-new-22k;
- build-new-86k。

构建结束后，检查 benchmark 程序是否生成：

```
find \  
~/secp256k1-exp/secp256k1-v041 \  
~/secp256k1-exp/secp256k1-v050 \  
-type f \  
-name bench \  
-executable
```

A.9 执行性能基准测试

A.9.1 单组 benchmark 命令

每次 benchmark 固定运行在逻辑 CPU 0，并将内部迭代次数设置为 50000：

```
SECP256K1_BENCH_ITERS=50000 \  
taskset -c 0 \  
./src/bench \  
    ecdsa_sign \  
    ec_keygen \  
    ecdsa_verify
```

其中：

- ecdsa_sign 测量 ECDSA 签名性能；
- ec_keygen 测量公钥生成性能；
- ecdsa_verify 作为负对照。

A.9.2 六配置交错重复测试

正式实验使用 12 轮交错运行，每种配置在六个执行位置中各出现两次。以下脚本为核心运行逻辑：

```
#!/usr/bin/env bash
set -euo pipefail

OUT="$HOME/secp256k1-exp/results-cross-version/raw"
rm -rf "$OUT"
mkdir -p "$OUT"

orders=(
    "old32 old64 old512 new2 new22 new86"
    "old64 old512 new2 new22 new86 old32"
    "old512 new2 new22 new86 old32 old64"
    "new2 new22 new86 old32 old64 old512"
    "new22 new86 old32 old64 old512 new2"
    "new86 old32 old64 old512 new2 new22"
    "new86 new22 new2 old512 old64 old32"
    "new22 new2 old512 old64 old32 new86"
    "new2 old512 old64 old32 new86 new22"
    "old512 old64 old32 new86 new22 new2"
    "old64 old32 new86 new22 new2 old512"
    "old32 new86 new22 new2 old512 old64"
)

bench_path() {
    case "$1" in
        old32)
            echo "$HOME/secp256k1-exp/secp256k1-v041/build-old-p2/src/bench"
            ;;
        old64)
            echo "$HOME/secp256k1-exp/secp256k1-v041/build-old-p4/src/bench"
            ;;
        old512)
            echo "$HOME/secp256k1-exp/secp256k1-v041/build-old-p8/src/bench"
            ;;
        new2)
            echo "$HOME/secp256k1-exp/secp256k1-v050/build-new-2k/src/bench"
            ;;
    esac
}
```

```
new22)
    echo "$HOME/secp256k1-exp/secp256k1-v050/build-new-22k/src/bench"
    ;;
new86)
    echo "$HOME/secp256k1-exp/secp256k1-v050/build-new-86k/src/bench"
    ;;
esac
}

for repeat in $(seq 1 12)
do
    order="${orders[$((repeat - 1))]}"
    position=0

    for case_id in $order
    do
        position=$((position + 1))
        bench="$(bench_path "$case_id")"
        outfile="$OUT/${case_id}-run-${repeat}.txt"

        {
            echo "# case_id=${case_id}"
            echo "# repeat=${repeat}"
            echo "# position=${position}"
            echo "# cpu_affinity=0"
            echo "# bench_iters=50000"
            echo "# timestamp=$(date -Iseconds)"
            echo

            SECP256K1_BENCH_ITERS=50000 \
            taskset -c 0 \
            "$bench" \
                ecdsa_sign \
                ec_keygen \
                ecdsa_verify
        } 2>&1 | tee "$outfile"

        sleep 2
    done
done
```

```
sleep 5
done
```

正式实验应生成：

$$6 \times 12 = 72$$

个原始结果文件。

检查文件数量：

```
find ~/secp256k1-exp/results-cross-version/raw \
-type f \
-name '*-run-*.txt' \
| wc -l
```

预期输出为：

72

检查每种配置的重复次数：

```
for case_id in old32 old64 old512 new2 new22 new86
do
    printf "%-7s: " "$case_id"

    find ~/secp256k1-exp/results-cross-version/raw \
        -type f \
        -name "${case_id}-run-*.txt" \
        | wc -l
done
```

每种配置均应输出 12。

A.9.3 记录编译产物大小

使用 stat 记录 benchmark 可执行文件和静态库大小：

```
stat -c '%s' build-directory/src/bench
stat -c '%s' build-directory/lib/libsecp256k1.a
```

对于不同构建环境，静态库的实际路径可能不同，可使用以下命令查找：

```
find build-directory \
-type f \
-name 'libsecp256k1.a'
```

最终将每种配置的表大小、平均运行时延和静态库大小统一汇总，用于计算性能提升百分比并绘制时间-空间关系图。